

팍리스 실무형 일경험 프로그램

SDV 아키텍처 기반 자율주행 통합 시스템 구현 실습



8일차

05 HPC Zone – 계기반 App 구현

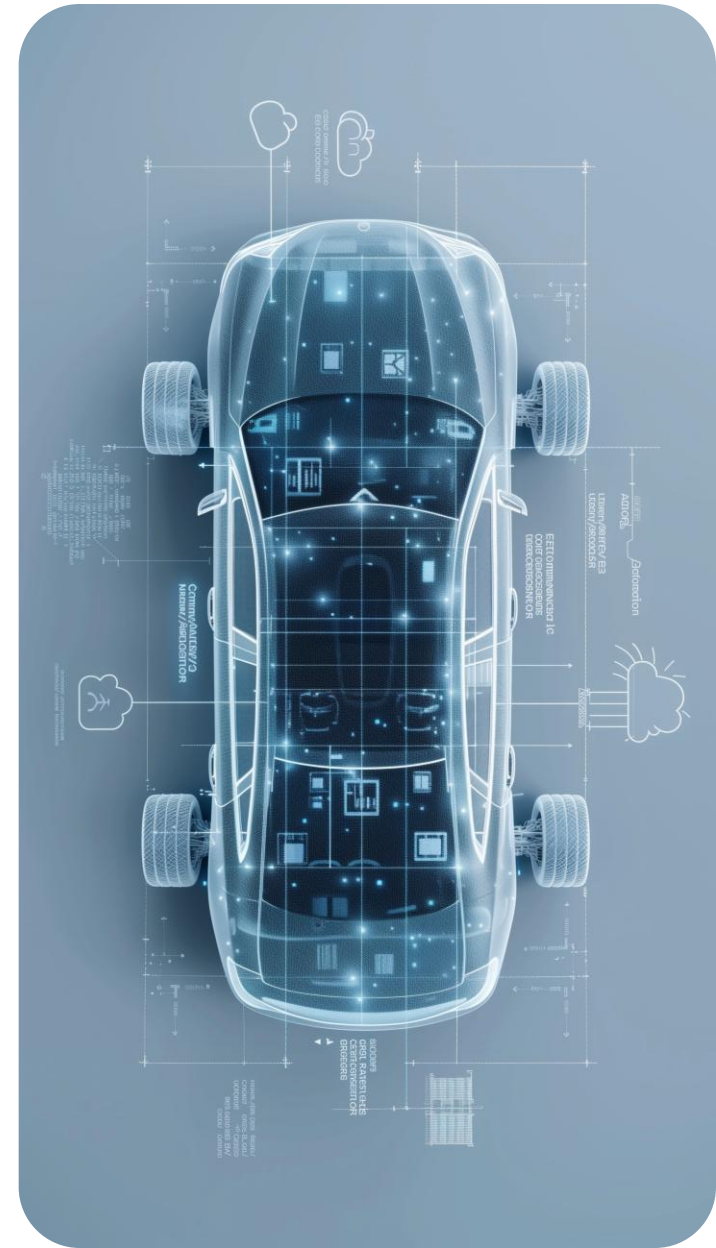
Telechips TOPST



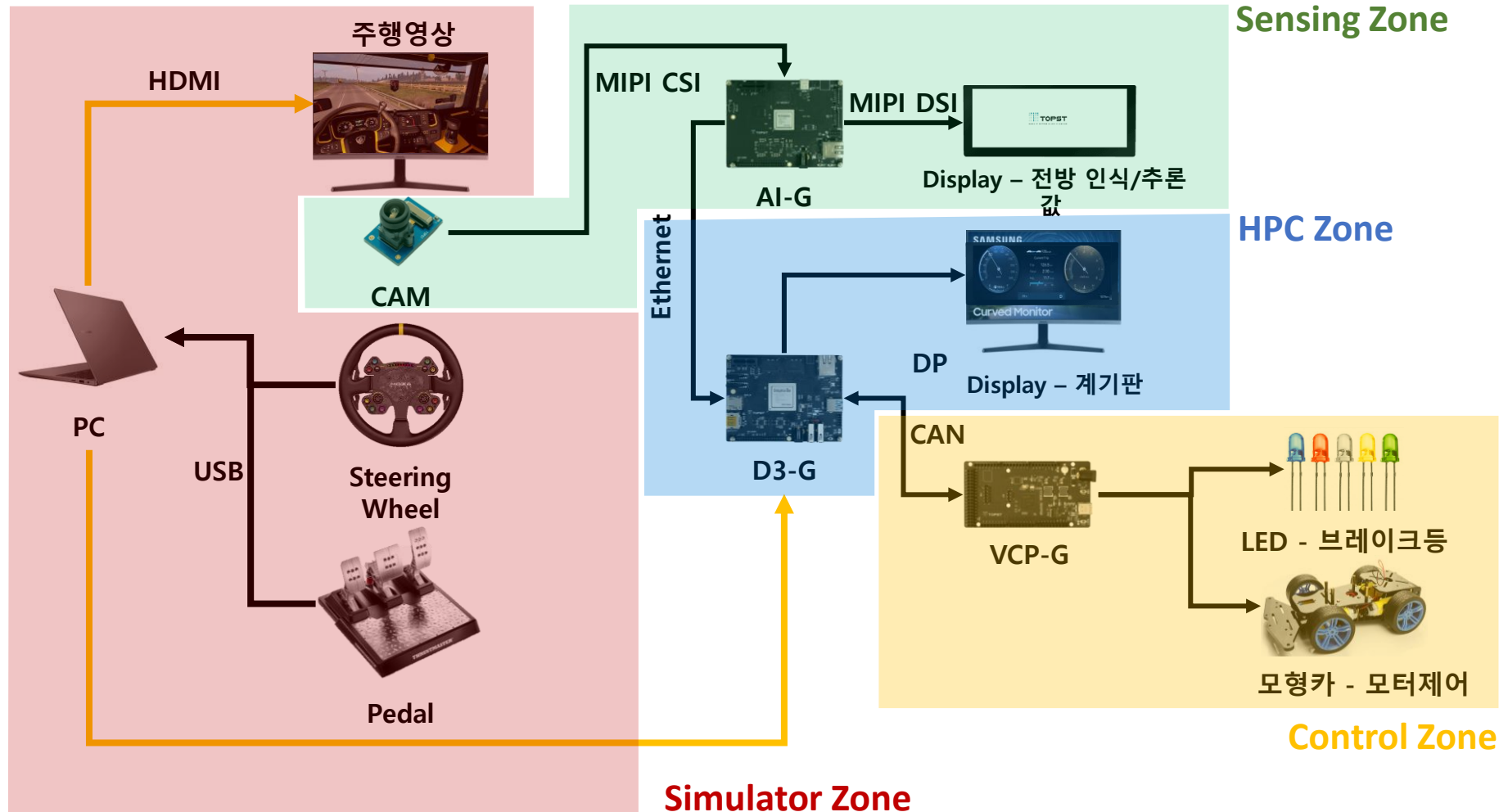
목차

Table of Contents

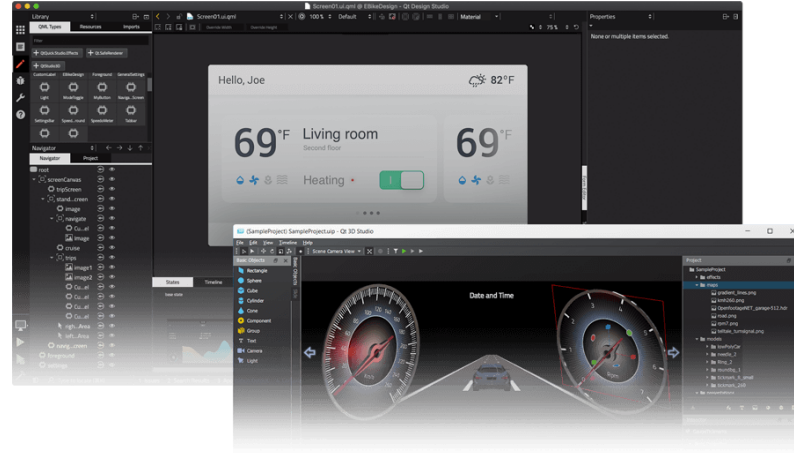
QT 소개	01
QT 환경 설정	02
Dashboard 코드 작성	03
동작 확인	04



Zonal 기반 차량 인터랙션 시스템-전체 시나리오



QT 소개



- 크로스플랫폼 앱, UI 프레임워크로 C++과 QML을 사용해 임베디드, 모바일에 주로 사용
- 전통 위젯과 GPU 가속의 QT Quick/QML을 제공
- 버튼, 클릭과 같은 이벤트를 시그널 / 슬롯으로 간단히 연결
- 장점 : 크로스 플랫폼, 빠른 UI 개발, 풍부한 라이브러리 지원

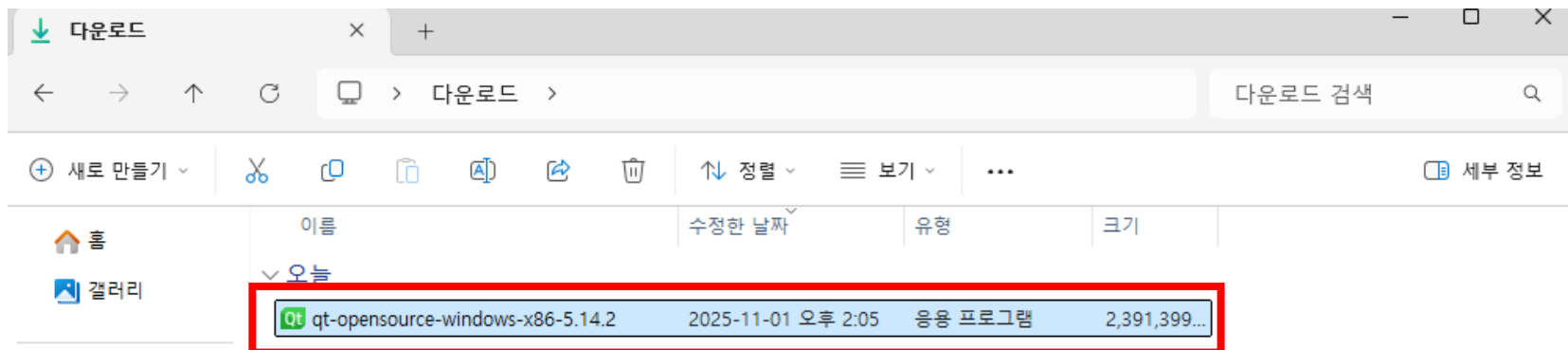
QT 환경 설정

- QT creator 설치
 - 링크 : [Index of /archive/qt/5.14/5.14.2](https://www.qt.io/archive/qt/5.14/5.14.2)

Name	Last modified	Size	Metadata
↑ Parent Directory		-	
submodules/	31-Mar-2020 09:27	-	
single/	31-Mar-2020 10:10	-	
qt-opensource-windows-x86-5.14.2.exe	31-Mar-2020 10:18	2.3G	Details
qt-opensource-mac-x64-5.14.2.dmg	31-Mar-2020 10:16	2.6G	Details
qt-opensource-linux-x64-5.14.2.run	31-Mar-2020 10:14	1.2G	Details
md5sums.txt	31-Mar-2020 10:32	207	Details

QT 환경 설정

- QT creator 설치
 - Qt 설치 파일 실행



QT 환경 설정

- QT creator 설치
 - Qt 계정 생성 후 Next 선택



Qt 5.14.2 Setup

Qt Account – Your unified login to everything Qt

Please log in to Qt Account

Login Email

Password

[Forgot password?](#)

Need a Qt Account?

Sign-up Valid email address

Password

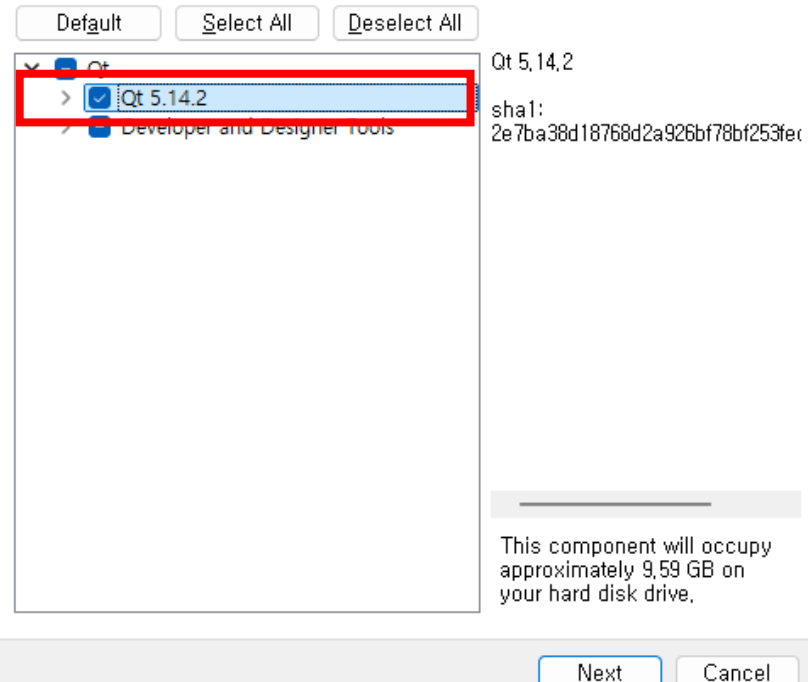
Confirm Password

☐ I accept the [service terms](#).

QT 환경 설정

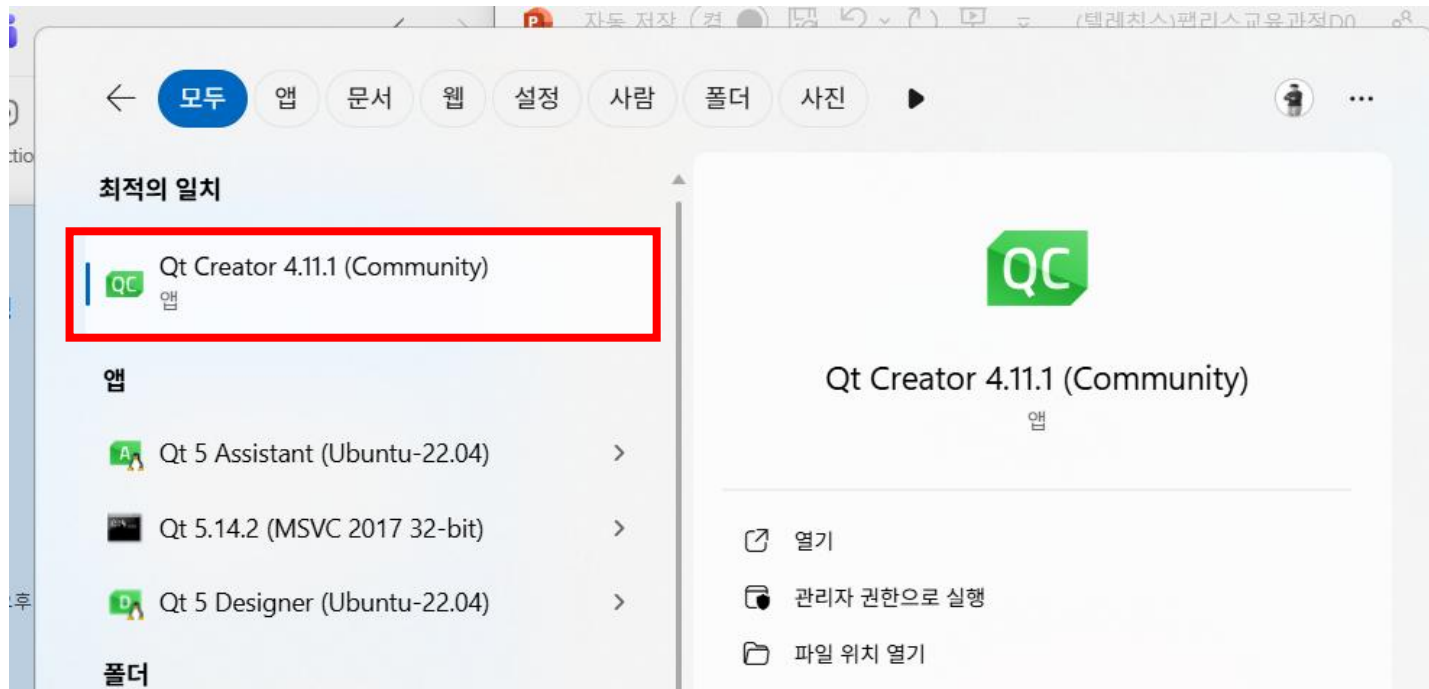
- QT creator 설치
 - Qt 5.14.2 활성화 후 Next 선택하여 설치 진행

Please select the components you want to install.



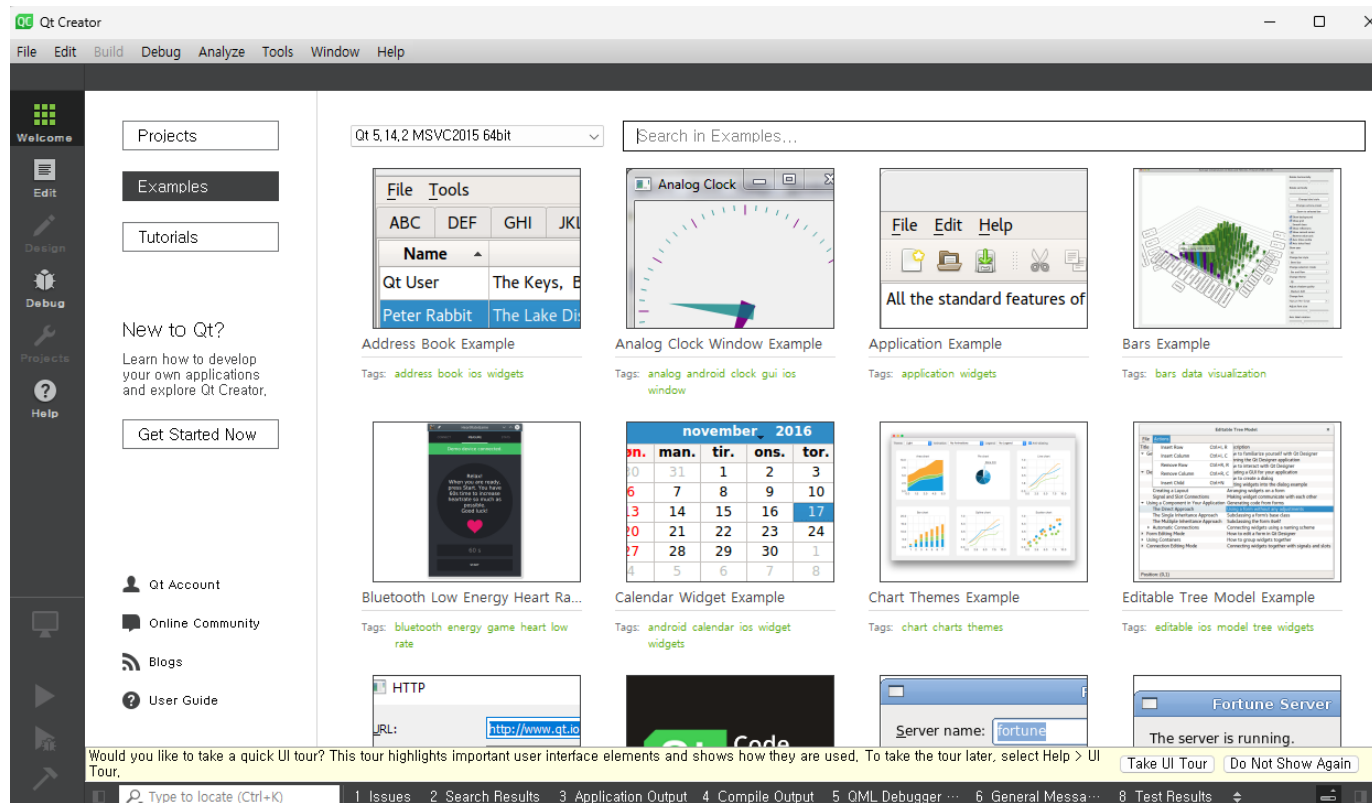
QT 환경 설정

- QT creator 실행



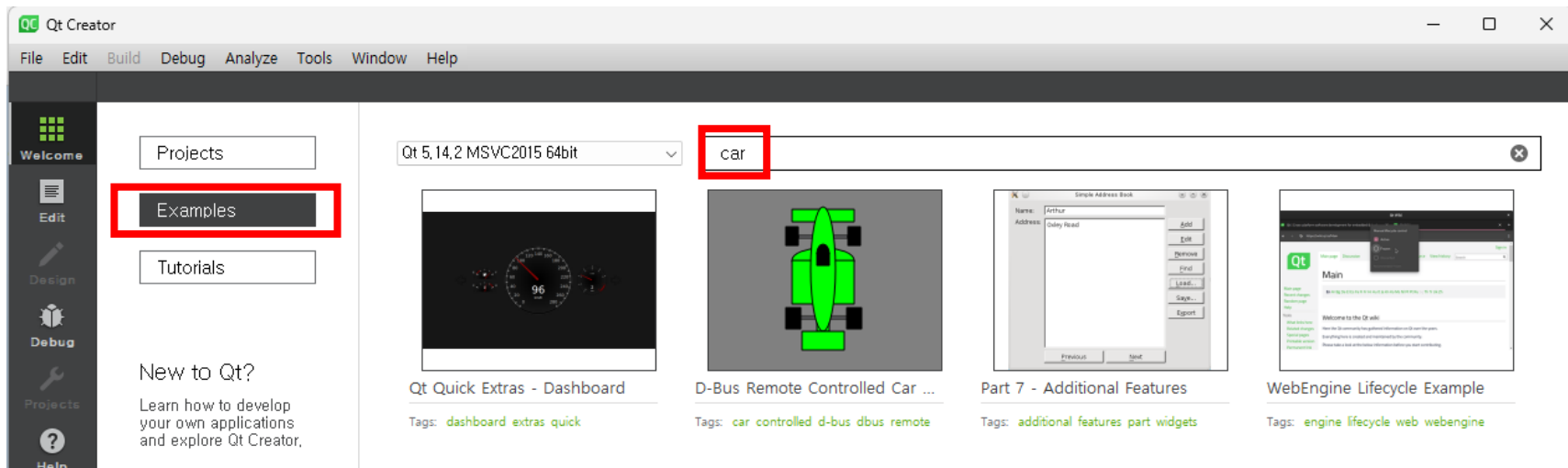
QT 환경 설정

- QT creator 실행 화면



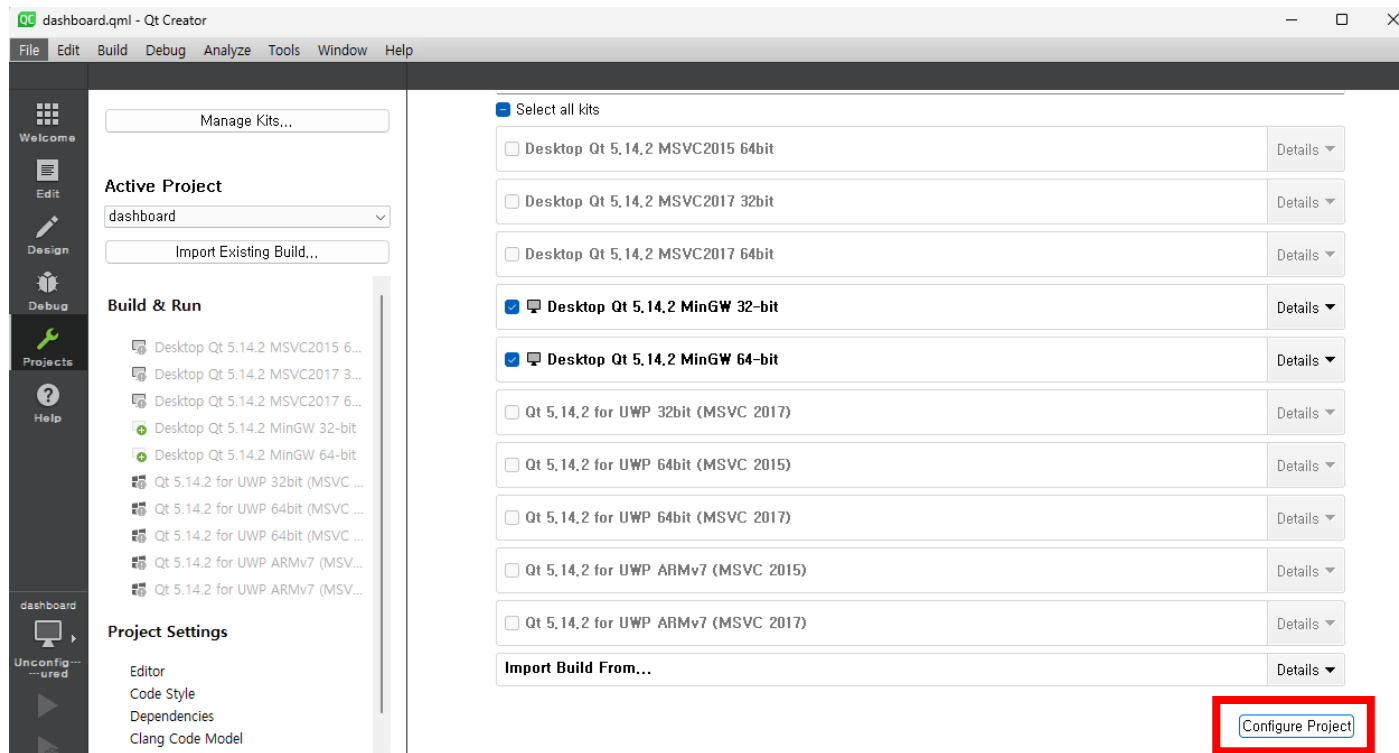
Dashboard 코드 작성

- Examples 탭에서 car 검색 후 dashboard 선택



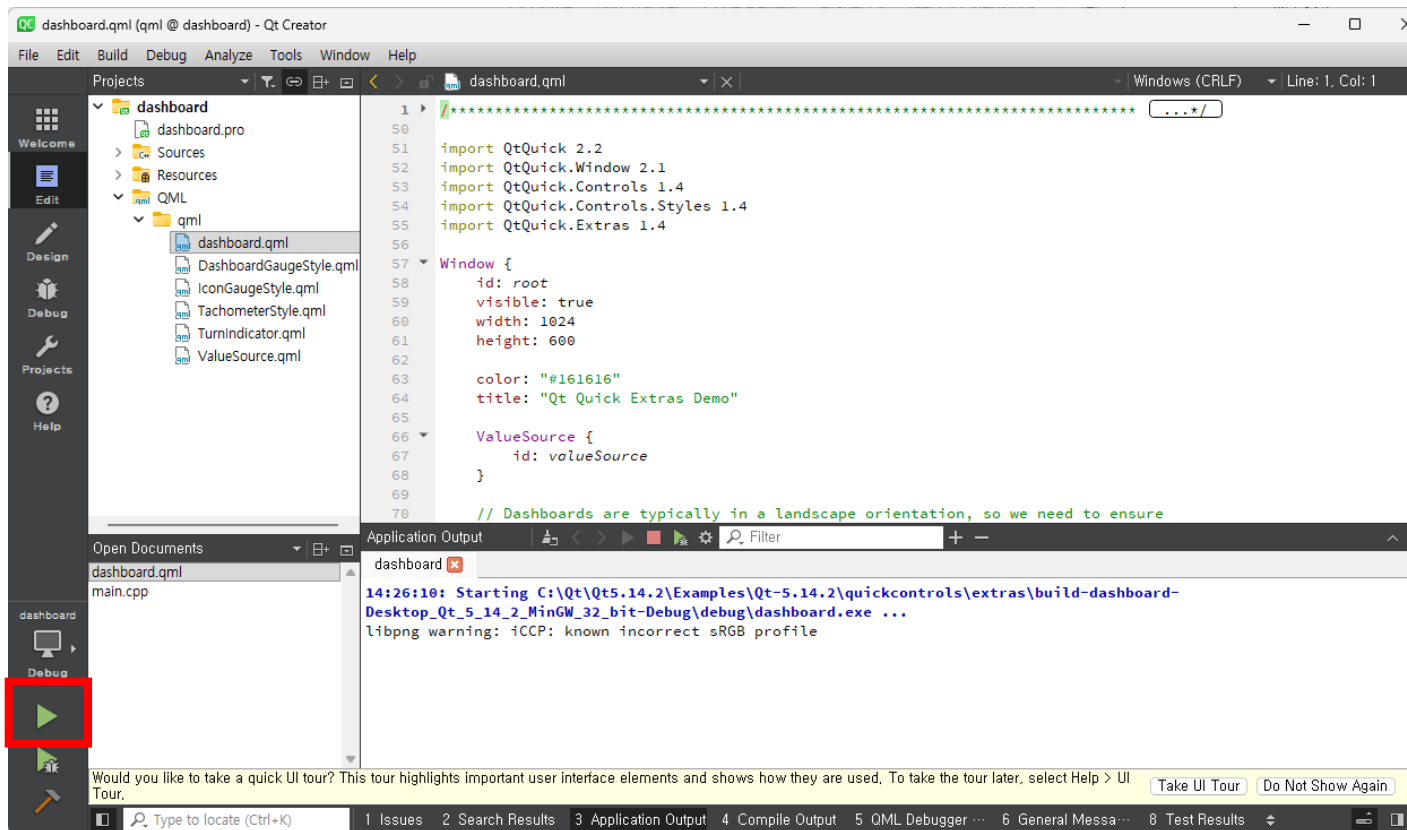
Dashboard 코드 작성

- dashborad 예제 실행



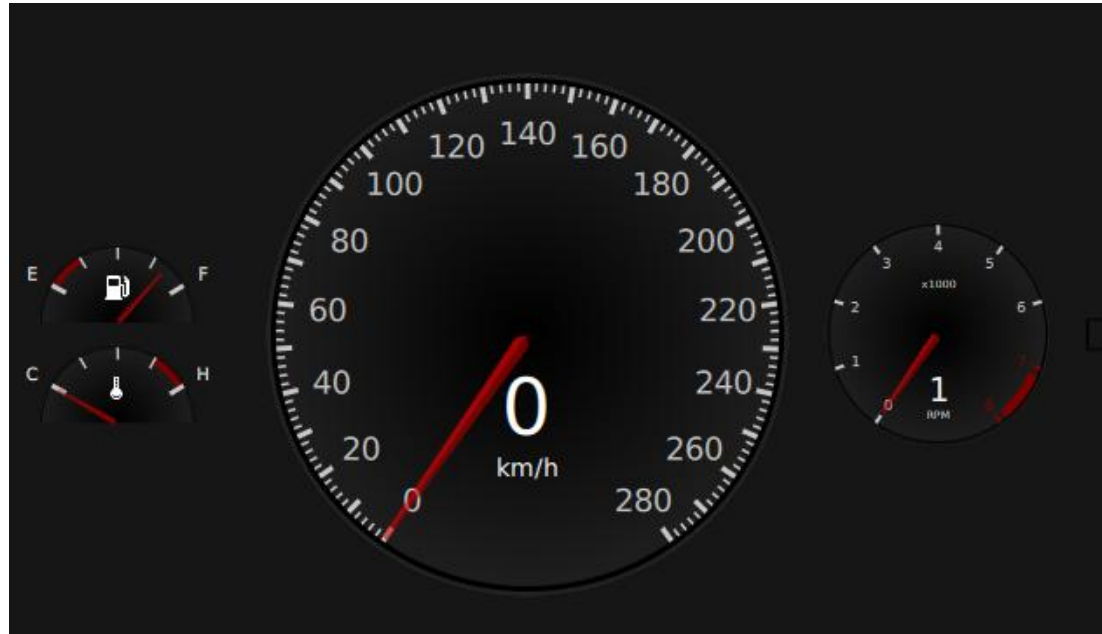
Dashboard 코드 작성

- dashborad 예제 실행



Dashboard 코드 작성

- dashborad 예제 실행



Dashboard 코드 작성

- dashborad와 연동을 위한 소스 수정
 - Dashboard.pro 파일 수정

```
TEMPLATE = app
TARGET = dashboard
INCLUDEPATH += .
QT += quick qml network gui

SOURCES += \
    main.cpp

RESOURCES += \
    dashboard.qrc

OTHER_FILES += \
    qml/dashboard.qml \
    qml/DashboardGaugeStyle.qml \
    qml/IconGaugeStyle.qml \
    qml/TachometerStyle.qml \
    qml/TurnIndicator.qml \
    qml/ValueSource.qml
```

Dashboard 코드 작성

- dashborad와 연동을 위한 소스 수정
 - main.cpp 파일 수정

```
#include <QGuiApplication>
#include <QQmlApplicationEngine>
#include <QQmlContext>
#include <QTcpSocket>
#include <QByteArray>
#include <QDebug>
#include <QtGui/QtFont>
#include <QtGui/QtFontDatabase>
#include <QJsonDocument>
#include <QJsonObject>
#include <QSurfaceFormat>
#include <QGuiApplication>
#include <QQmlApplicationEngine>
#include <QQuickView>
```

```
class TcpClient : public QObject {
    Q_OBJECT
public:
    explicit TcpClient(QObject* parent=nullptr) :
    QObject(parent) {
        connect(&socket, &QTcpSocket::connected, this, []{
            qDebug() << u8"서버에 연결 성공!";
        });
        connect(&socket, &QTcpSocket::disconnected, this,
        []{
            qDebug() << u8"서버와 연결이 끊어졌습니다.";
        });
        connect(&socket, &QTcpSocket::readyRead, this,
        &TcpClient::onReadyRead);
        socket->connectToHost("127.0.0.1", 9999);
    }

signals:
    void telemetryUpdated(double rpm, double speedKmh,
    double fuelL, double steering);
```

Dashboard 코드 작성

- dashborad와 연동을 위한 소스 수정
 - main.cpp 파일 수정

```
private slots:
    void onReadyRead() {
        buf_.append(socket.readAll());
        int idx;
        while ((idx = buf_.indexOf('\n')) != -1) {
            const QByteArray line = buf_.left(idx);
            buf_.remove(0, idx + 1);
            handleLine(line.trimmed());
        }
    }

private:
    void handleLine(const QByteArray& line) {
        if (line.isEmpty()) return;
        if (line[0] != '{' && line[0] != '[') {
            qDebug().noquote() << "RX(non-JSON):"
<< line;
            return;
        }
    }
}
```

```
QJsonParseError err;
    const QJsonDocument doc =
QJsonDocument::fromJson(line, &err);
    if (err.error != QJsonParseError::NoError
|| !doc.isObject()) {
        qDebug() << "JSON parse error:"
<< err.errorString() << "data=" << line.left(120);
        return;
    }

    const QJsonObject o = doc.object();
    const double rpm    = o.value("rpm").toDouble();
    const double speed  = o.value("speed_kmh").toDouble();
    const double fuel   = o.value("fuel_l").toDouble();
    const double steer  = o.value("steering").toDouble();
    emit telemetryUpdated(rpm, speed, fuel, steer);
}

QTcpSocket socket;
QByteArray buf_;
};
```

Dashboard 코드 작성

- dashborad와 연동을 위한 소스 수정
 - main.cpp 파일 수정

```
int main(int argc, char *argv[])
{
    QGuiApplication app(argc, argv);
    QSurfaceFormat format;
    format.setRenderableType(QSurfaceFormat::OpenGL);
    format.setVersion(2, 0);
    QSurfaceFormat::setDefaultFormat(format);

    QFontDatabase::addApplicationFont(":/fonts/DejaVuSans.ttf");

    app.setFont(QFont("DejaVu Sans"));
    TcpClient client;
    QQmlApplicationEngine engine;
    engine.rootContext()->setContextProperty("tcpClient", &client);
    engine.load(QUrl(QStringLiteral("qrc:/qml/dashboard.qml")));
    if (engine.rootObjects().isEmpty())
        return -1;
    return app.exec();
}

#include "main.moc"
```

Dashboard 코드 작성

- dashborad와 연동을 위한 소스 수정
 - dashboard.qml파일 수정

```
import QtQuick 2.2
import QtQuick.Window 2.1
import QtQuick.Controls 1.4
import QtQuick.Controls.Styles 1.4
import QtQuick.Extras 1.4

Window {
    id: root
    visible: true
    width: 1024
    height: 600

    color: "#161616"
    title: "Qt Quick Extras Demo"

    ValueSource {
        id: valueSource
    }
}
```

```
Item {
    id: container
    width: root.width
    height: Math.min(root.width, root.height)
    anchors.centerIn: parent

    Row {
        id: gaugeRow
        spacing: container.width * 0.02
        anchors.centerIn: parent

        TurnIndicator {
            id: leftIndicator
            anchors.verticalCenter:
parent.verticalCenter
            width: height
            height: container.height * 0.1 -
gaugeRow.spacing

            direction: Qt.LeftArrow
            on: valueSource.turnSignal ==
Qt.LeftArrow
        }
    }
}
```

Dashboard 코드 작성

- dashborad와 연동을 위한 소스 수정
 - dashboard.qml파일 수정

```
Item {
    width: height
    height: container.height * 0.25 -
    gaugeRow.spacing
    anchors.verticalCenter:
    parent.verticalCenter

    CircularGauge {
        id: fuelGauge
        value: valueSource.fuel
        maximumValue: 1
        y: parent.height / 2 - height / 2 -
        container.height * 0.01
        width: parent.width
        height: parent.height * 0.7

        style: IconGaugeStyle {
            id: fuelGaugeStyle

            icon: "qrc:/images/fuel-icon.png"
            minWarningColor: Qt.rgb(0.5, 0,
0, 1)
```

```
tickmarkLabel: Text {
    color: "white"
    visible: styleData.value ===
0 || styleData.value === 1
    font.pixelSize:
    fuelGaugeStyle.toPixels(0.225)
    text: styleData.value === 0 ?
"E" : (styleData.value === 1 ? "F" : "")
    }
}
```

Dashboard 코드 작성

- dashborad와 연동을 위한 소스 수정
 - dashboard.qml파일 수정

```
CircularGauge {
    value: valueSource.temperature
    maximumValue: 1
    width: parent.width
    height: parent.height * 0.7
    y: parent.height / 2 + container.height * 0.01

    style: IconGaugeStyle {
        id: tempGaugeStyle

        icon: "qrc:/images/temperature-icon.png"
        maxWarningColor: Qt.rgb(0.5, 0, 0, 1)

        tickmarkLabel: Text {
            color: "white"
            visible: styleData.value === 0 ||
styleData.value === 1
            font.pixelSize:
tempGaugeStyle.toPixels(0.225)
            text: styleData.value === 0 ? "C" :
(styleData.value === 1 ? "H" : "")
        }
    }
}
```

```
CircularGauge {
    id: speedometer
    value: valueSource.kph
    anchors.verticalCenter:
parent.verticalCenter
    maximumValue: 280
    width: height
    height: container.height * 0.5

    style: DashboardGaugeStyle {}
}
CircularGauge {
    id: tachometer
    width: height
    height: container.height * 0.25 -
gaugeRow.spacing
    value: valueSource.rpm * 0.001
    maximumValue: 8
    anchors.verticalCenter:
parent.verticalCenter

    style: TachometerStyle {}
}
```

Dashboard 코드 작성

- dashborad와 연동을 위한 소스 수정
 - dashboard.qml파일 수정

```
TurnIndicator {
    id: rightIndicator
    anchors.verticalCenter:
parent.verticalCenter
    width: height
    height: container.height * 0.1 -
gaugeRow.spacing

    direction: Qt.RightArrow
    on: valueSource.turnSignal ==

Qt.RightArrow
}

}

}
```

Dashboard 코드 작성

- dashborad와 연동을 위한 소스 수정
 - ValueSource.qml파일 수정

```
import QtQuick 2.2
import QtQuick.Window 2.1
import QtQuick.Controls 1.4
import QtQuick.Controls.Styles 1.4
import QtQuick.Extras 1.4
Item {
    id: valueSource
    property bool useRealData: true
    property real kph: 0
    property real rpm: 1
    property real fuel: 0.85
    Connections {
        target: tcpClient
        onTelemetryUpdated: function(rpm0to6, speedKmh,
fuelL, steering) {
            if (useRealData) {
                kph = speedKmh
                rpm = rpm0to6
                fuel = Math.max(0, Math.min(1, fuelL /
1400.0))
            }
        }
    }
}
```

```
SequentialAnimation {
    running: !useRealData
    loops: Animation.Infinite

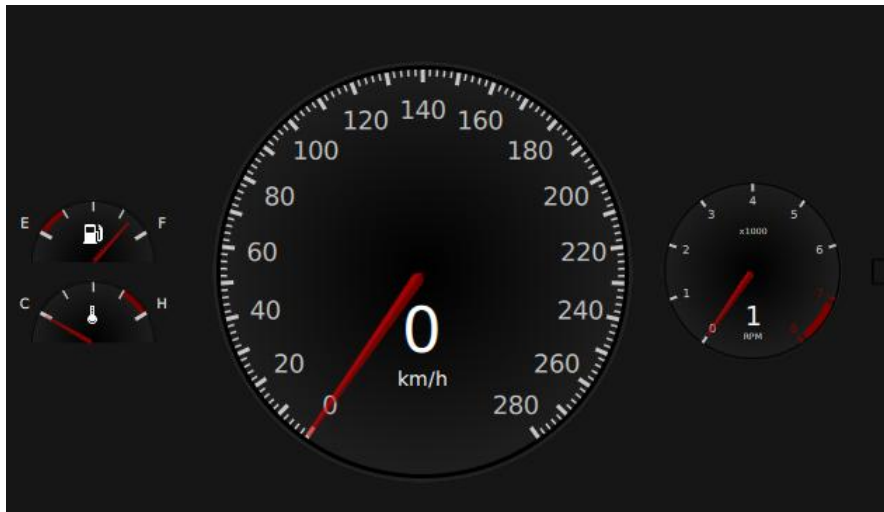
}
}
```

동작 확인

- dashborad 빌드(D3G-G Terminal)
 - \$ cd Fabless_Education_blink/Step3/dashboard
 - \$ sudo chmod 755 dashboard
 - \$ sudo make clean
 - \$ sudo qmake
 - \$ sudo make

동작 확인

- QT_PLATFORM 설정 및 실행(D3-G Terminal)
 - `vi ~/.bashrc`
맨 아랫 줄에 `export QT_QPA_PLATFORM=wayland` 추가
 - `source ~/.bashrc`
 - `./dashboard`



Dashboard 연동을 위한 코드 수정

- 파일 수정 권한 부여
- `$ cd Fabless_Education_blink/Step3`
- `$ sudo chmod 777 *`

Dashboard 연동을 위한 코드 수정

- 편집기 실행
 - \$ geany
- 상단의 File탭에 Open을 통해 Fabless_Education/Step3/Data_parsing_QT.py 열기
- QT와 연동을 위한 추가 기능 작성 (library / variables)

```
@dataclass
class Config:
    """Application Configuration"""
    IPC_PATH: str = "/dev/tcc_ipc_micom"
    TELEMETRY_URL: str = "http://192.168.137.1:25555/api/ets2/telemetry"
    SERVER_HOST: str = "0.0.0.0"
    SERVER_PORT: int = 9999

    # Frequencies (Hz)
    POLL_HZ: int = 60
    SEND_HZ: int = 60

    # Task Frequencies (Hz)
    TASK_HZ: Dict[str, int] = field(default_factory=lambda: {
        "break": 5, "head": 5, "turn": 2,
        "speed": 10, "fuel": 1, "wheel": 10
    })

    IDLE_RPM: int = 552
    BLINK_INTERVAL: float = 1
```

Dashboard 연동을 위한 코드 수정

- QT와 연동을 위한 추가 기능 작성 (function)

```
def _worker_tcp_server(self):
    """Thread: TCP Server for Qt GUI"""
    srv = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

    try:
        srv.bind((self.cfg.SERVER_HOST, self.cfg.SERVER_PORT))
        srv.listen(1)
        srv.settimeout(1.0) # Allow checking self.running periodically
        print(f"[TCP] Server listening on {self.cfg.SERVER_HOST}:{self.cfg.SERVER_PORT}")
    except Exception as e:
        print(f"[TCP] Bind Failed: {e}")
        return

    send_keys = {"rpm", "speed_kmh", "fuel_l", "steering"} # "steering" in JSON vs "steer" in dict
    period = 1.0 / self.cfg.SEND_HZ
```

Dashboard 연동을 위한 코드 수정

- QT와 연동을 위한 추가 기능 작성 (function)

```
while self.running.is_set():
    try:
        conn, addr = srv.accept() # Blocks for 1.0s
        print(f"[TCP] Client connected: {addr}")

        with conn:
            conn.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
            while self.running.is_set():
                iter_start = time.monotonic()

                with self.lock:
                    # Remap keys for Qt Client
                    payload = {k: self.telemetry_data.get(k if k != "steering" else "steer", 0)
                               for k in send_keys}

                msg = json.dumps(payload, separators=(",", ":")) + "\n"
                conn.sendall(msg.encode("utf-8"))
                elapsed = time.monotonic() - iter_start
                if period > elapsed:
                    time.sleep(period - elapsed)

            except socket.timeout:
                continue # Loop back to check running state
            except Exception as e:
                print(f"[TCP] Connection Error: {e}")

    srv.close()
```

Dashboard 연동을 위한 코드 수정

- QT와 연동을 위한 추가 기능 작성 (function)

```
def start(self):
    """Start all worker threads"""
    workers = [
        self._worker_http_poller,
        self._worker_can_sender,
        self._worker_tcp_server
    ]

    for func in workers:
        t = threading.Thread(target=func, daemon=True, name=func.__name__)
        t.start()
        self.threads.append(t)

    print("[SYS] All threads started.")

    # Main thread loop
    try:
        while True:
            time.sleep(1)
    except KeyboardInterrupt:
        print("\n[SYS] KeyboardInterrupt detected.")
```

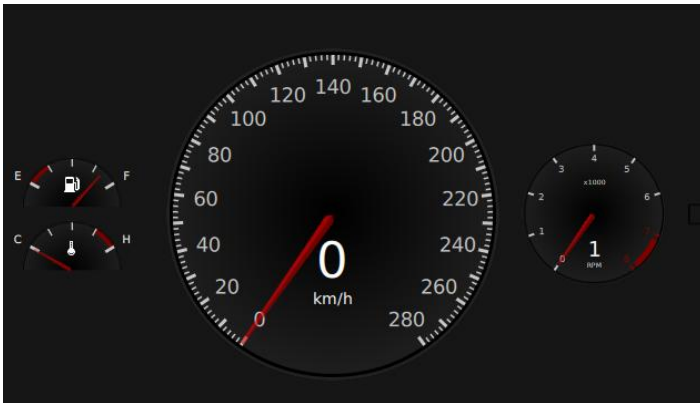
```
if __name__ == "__main__":
    # Setup Argument Parser
    parser = argparse.ArgumentParser(description="ETS2 Telemetry to IPC Bridge")
    parser.add_argument("--ip", default="10.10.129.118", help="ETS2 Server IP")
    args = parser.parse_args()

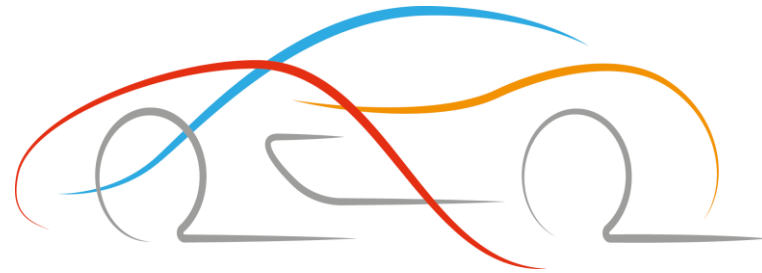
    # Create Config
    config = Config()
    config.TELEMETRY_URL = f"http://{args.ip}:25555/api/ets2/telemetry"

    # Run Application
    with ETSTelemetryMonitor(config) as app:
        app.start()
```

동작 확인

- D3-G & 계기판 연동 확인
- `$ cd /Fabless_Education/Step3`
- `$ sudo python3 Data_parsing_QT.py`
- 터미널 창 하나 더 열기
- `$ cd /Fabless_Education/Step3/dashboard`
- `$ ./dashboard`





Thank You